

Edge-Conditioned One-Step Image Generation via MeanFlow

Extending MeanFlow from Class-Conditional to Spatial Conditioning

Computer Vision Practice with Deep Learning (Fall 2025)

Student: 7114056082 柯皓哲

Date: December 21, 2025

The Generative Speed Bottleneck

Why do we need one-step generation?

State-of-the-Art Quality, but High Cost

- Diffusion models and Flow Matching have achieved SOTA image quality.
- **Critical Limitation:** Inference requires iterative solving of ODEs (typically 20-50 steps).
- High latency prevents real-time applications and efficient edge deployment.

Limitations of Existing Acceleration Methods

- Distillation: Requires training a heavy teacher model first.
- Consistency Models: Often involve complex curriculum learning and unstable training.
- Goal: We seek a principled, training-stable framework for single-step generation.

MeanFlow: A Principled Framework

Solving generation with Average Velocity

Core Concept: Average vs. Instantaneous Velocity

- Traditional Flow Matching learns the **instantaneous velocity** v_t (tangent to the path).
- MeanFlow proposes learning the **average velocity** u between time t and r .

The One-Step Mechanism

- By definition, average velocity connects two points directly:

$$z_0 = z_1 - u(z_1, 0, 1)$$

- This enables mathematically exact one-step generation without numerical integration.

Key Advantages

- No teacher model required (Training from scratch).
- No complex curriculum learning.
- State-of-the-art performance (FID 3.43 on ImageNet).

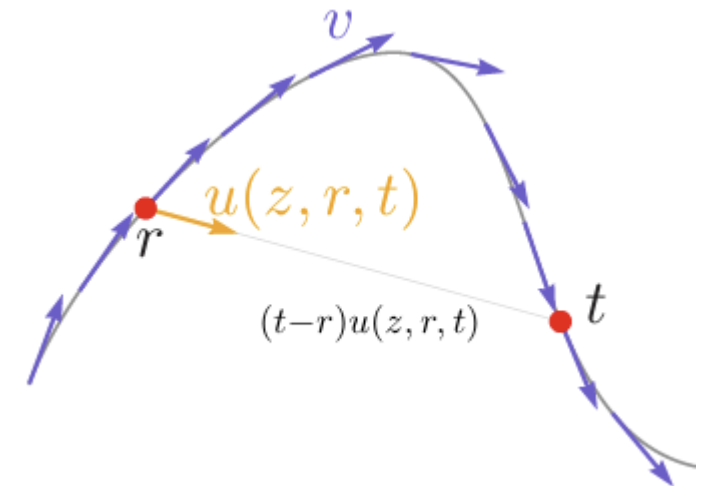


Fig 1: The field of average velocity

Project Goal: From Global to Spatial Conditioning

Bridging the gap for controllable generation

Current Limitation of MeanFlow

- The original paper only explores **Class-Conditional** generation.
- Condition is a global vector (class embedding), lacking dense spatial control.



The Gap: Spatial Control

Our Objective: Edge-to-Image Generation

- Extend MeanFlow to accept high-resolution **Edge Maps** (256×256).
- Enable precise structural control (e.g., ControlNet-like applications).

Why is this important?

- Spatial conditioning is essential for practical editing workflows (sketch-to-image).
- Validates the robustness of the MeanFlow framework on complex, dense conditions.

Technical Challenges

Barriers to implementing Edge-Conditioned MeanFlow

Resolution Mismatch

- **Issue:**
Edge maps are in pixel space (256×256), while the Latent Diffusion backbone operates in compressed latent space (32×32).
- **Impact:**
Direct downsampling causes loss of high-frequency structural details.

Edge Quality & Semantics

- **Issue:**
Traditional operators (Canny/Sobel) capture noisy textures (e.g., fur) as edges.
- **Impact:**
Too much noise confuses the flow matching objective; the model fails to distinguish structure from texture.

Training Stability

- **Issue:**
MeanFlow relies on Jacobian-Vector Product (JVP) optimization, which is sensitive to gradient variance.
- **Impact:**
Injecting strong spatial conditions can destabilize the delicate JVP learning process.

Contributions

Summary of work delivered

1. Independent MeanFlow Implementation [Re-implementation]

- Full implementation of the MeanFlow framework (JVP optimization, Adaptive Loss, Training-time CFG) based on mathematical derivation, without official code.

2. Multi-scale Edge Encoder [Novelty]

- By definition, average velocity connects two points directly:
- This enables mathematically exact one-step generation without numerical integration.

3. Semantic Edge Preprocessing [Novelty]

- Adopted PiDiNet with a custom "Safe Mode" quantization strategy.
- Successfully filtered texture noise to provide clean semantic guidance for animal faces.

Background: Flow Matching

Constructing Straight Paths between Distributions

The Concept

- Flow Matching (FM) learns a velocity field v_t that transports a source distribution (noise) to a target distribution (data).
- FM explicitly constructs straight paths via linear interpolation:

$$z_t = (1 - t)x + t\epsilon$$

The Training Target

- The ground truth velocity is simply the direction pointing from data to noise:

$$v_t(z_t|x)$$

- The model minimizes the regression loss: $\mathbb{E} \|v_\theta(z_t, t) - v_t\|^2$.

Why is this still slow?

- Although the conditional paths are straight, the marginal vector field (what the network learns) is often curved.
- This curvature necessitates multi-step numerical integration (ODE solvers) during sampling.

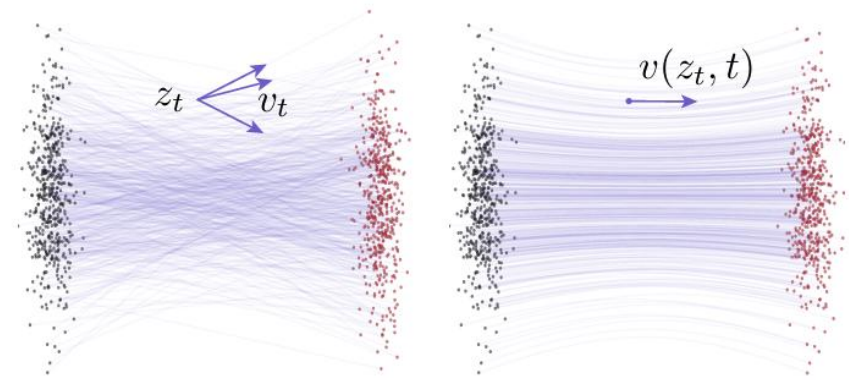


Fig 2: Velocity fields in Flow Matching

The Landscape of One-Step Generation

Comparing MeanFlow with Existing Paradigms

Comparison of Accelerated Generative Models

Method	Core Idea	Training Requirements	Limitations
Consistency Models	Enforce self-consistency of points on the trajectory $(f(x_t, t) = f(x_{t'}, t'))$.	Curriculum Learning (complex) or Distillation.	Unstable training; relies on heuristic constraints.
Shortcut Models	Add a specific loss to connect discrete time steps.	Double Constraints (Flow + Consistency).	Constraints are imposed on network behavior, not the field itself.
MeanFlow (Ours)	Learn the Average Velocity directly defined by calculus.	Standard Training (No curriculum, No teacher).	Purely mathematically derived; naturally supports 1-step.

MeanFlow Theory: Average Velocity

Bypassing integration via physics definition

Instantaneous Velocity v (Flow Matching)

- Describes the tangent direction at a specific time τ .
- Requires integration (stepping) to find the path: $z_t = z_r + \int v d\tau$.

Average Velocity u (MeanFlow)

- Defined as the total displacement divided by the time interval:

$$u(z_t, r, t) \triangleq \frac{1}{t - r} \int_r^t v(z_\tau, \tau) d\tau$$

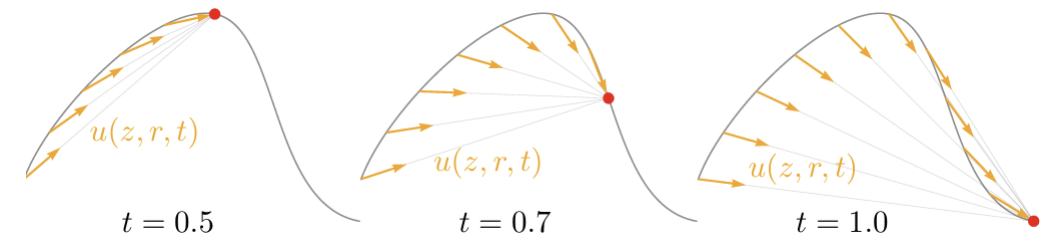


Fig 3: Average Velocity fields with time interval

The One-Step Advantage

- If the model predicts u correctly, we can jump from noise ($t = 1$) to data ($t = 0$) in a single step:

$$z_0 = z_1 - 1 \cdot u(z_1, 0, 1)$$

The MeanFlow Identity

Training without numerical integration

The Learning Objective

- Directly training on the integral definition is intractable.
- Differentiating the definition w.r.t. t yields the **MeanFlow Identity**:

$$\underbrace{u(z_t, r, t)}_{\text{average vel.}} = \underbrace{v(z_t, t)}_{\text{instant. vel.}} - (t - r) \underbrace{\frac{d}{dt} u(z_t, r, t)}_{\text{time derivative}}$$

Why is this brilliant?

- It relates the unknown average velocity u to the known instantaneous velocity v_t (Ground Truth).
- The total derivative $\frac{d}{dt} u$ can be computed efficiently via **Jacobian-Vector Product (JVP)**.

Implementation

- We use `torch.func.jvp` to compute the derivative term in a single forward-mode differentiation pass.

Training-time Classifier-Free Guidance

Baking guidance into the vector field

Traditional CFG (Inference-time)

- Combines conditional and unconditional predictions during sampling.
- **Cost:** Doubles the NFE (2 evaluations per step).

MeanFlow CFG (Training-time)

- We construct a new ground-truth field v^{cfg} that inherently contains guidance.
- Target Construction:

$$\tilde{v}_t = \omega \cdot v_t + (1 - \omega) \cdot u_{\theta}^{uncond}(z_t, t, t)$$

- **Benefit:** The model learns the guided trajectory directly. Sampling remains **1-NFE**.

Adaptation for Spatial Conditioning **[Contribution]**

- We implement condition dropping by replacing the Edge Map with a learned null tensor (or zeros).

Edge-Conditioned Architecture

Integrating spatial control into the U-Net backbone

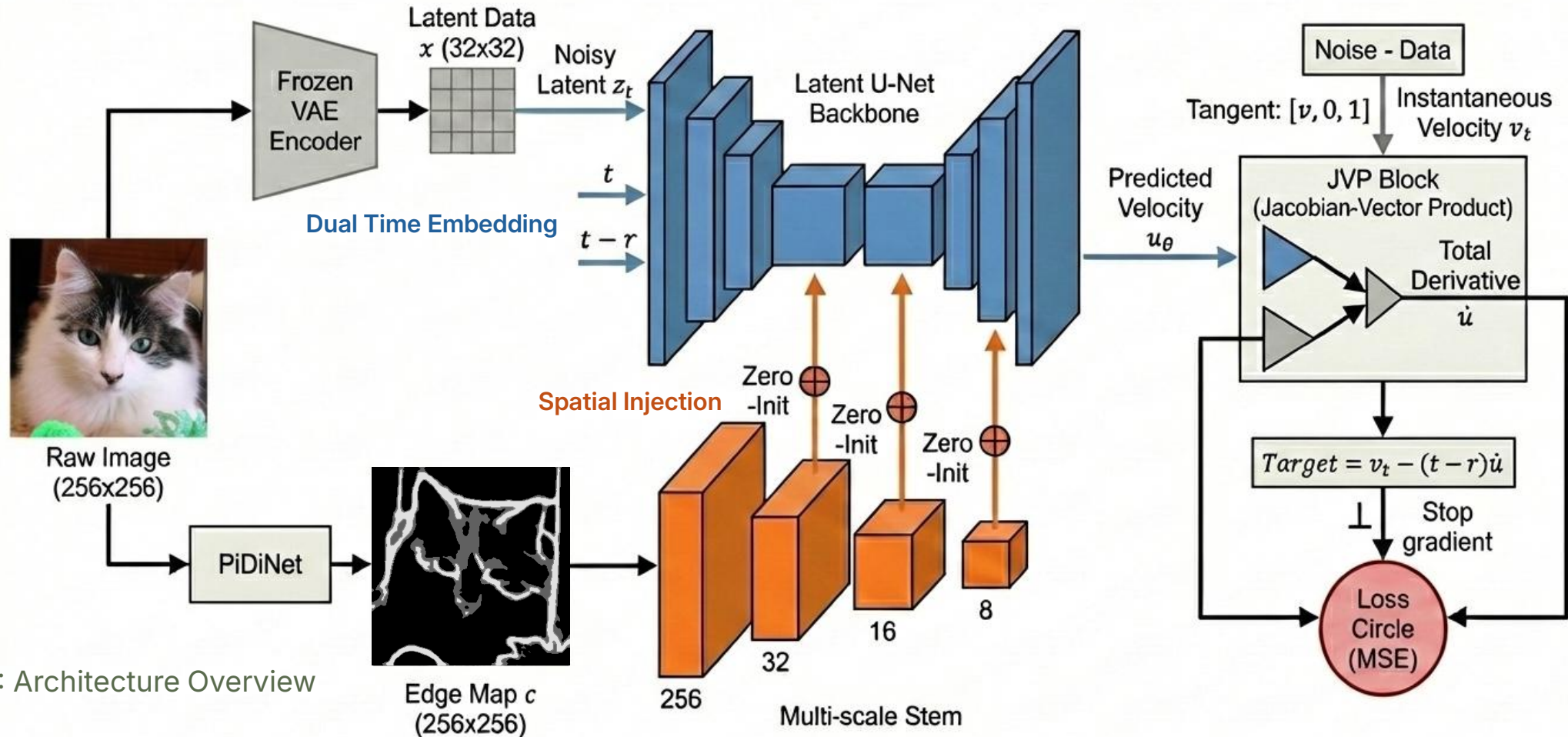


Fig 4: Architecture Overview

Multi-scale Edge Encoder

Bridging the resolution gap with Zero-Convolution

The Resolution Challenge

- Edge Map: 256×256 (High-frequency geometry).
- Latent Space: 32×32 (Compressed semantics).

Hierarchical Design [Novelty]

- **Stem:** Compresses $256\text{px} \rightarrow 32\text{px}$ via strided convolutions.
- **Multi-scale Features:** Extracts features at $32^2, 16^2, 8^2$ resolutions to match U-Net encoder levels.

Zero-Convolution Strategy

- The final layer of each scale is initialized to zero.
- **Effect:** Ensures the model starts training as an unconditional MeanFlow model, gradually learning to utilize edge cues without destabilization.

Semantic Edge Preprocessing

Robust edge detection for animal faces

Why not Canny/Sobel/Scharr?

- Traditional gradient operators capture all intensity changes, including fur textures and background noise.
- This confuses the flow matching objective, forcing the model to generate specific textures rather than general shapes.

PiDiNet with Safe Mode **[Contribution]**

- We employ PiDiNet (Pixel Difference Network) for semantic edge detection.
- Safe Mode Quantization: 8-level quantization + morphological cleaning removes hidden grayscale artifacts, providing clean structural guidance.



Raw image



Schar



PiDiNet

Fig 5: PiDiNet successfully filters out texture noise.

Implementation Details & Stability

Ensuring robust JVP optimization

JVP Stability Techniques

- **ChannelLayerNorm**: Replaces GroupNorm to ensure stable gradient computation across the channel dimension during JVP.
- **Target Clipping**: Clamps the regression target to $[-8, 8]$ to prevent gradient explosions from transient JVP spikes.

Training Configuration

- Dataset: AFHQ v2 (Cat, Dog, Wild).
- Optimizer: AdamW (LR=1e-4) with Cosine Annealing.
- Hardware: Single NVIDIA RTX 4070 (Showing accessibility).

Engineering the MeanFlow Core

Optimization strategy and stability design

Efficient JVP Optimization

- **Decision:** Instead of calculating the full Jacobian matrix (expensive), we compute the Jacobian-Vector Product (JVP) directly.
- **Implementation:** Leveraging `torch.func.jvp` for efficient forward-mode differentiation within the training loop.

ChannelLayerNorm for Stability

- **Problem:** Standard GroupNorm or BatchNorm can introduce instability when computing higher-order derivatives required by JVP.
- **Solution:** Implemented custom ChannelLayerNorm (statistics over channel dim only), ensuring robust gradient propagation.

ChannelLayerNorm for Stability

- **Decision:** Initialize the final output layers to zero.
- **Effect:** Ensures the initial predicted velocity field is near-zero, preventing large gradients from destabilizing the delicate JVP learning process early in training.

Engineering Spatial Conditioning

Handling high-res edges and guidance

Pixel-Space Downsampling vs. VAE Encoding

- **Challenge:** Why not use the pre-trained VAE to encode 256×256 edges?
 - VAEs are lossy and optimized for RGB textures, not sparse structural lines.
 - Encoding thin edges into VAE latent space results in **structural degradation** and artifacts.
- **Our Solution:** Keep edges in **Pixel Space** and use a custom **CNN-based Downsampler**.
 - Strided Convolutions learn to preserve high-frequency structural cues while matching the 32×32 latent resolution.

Experimental Setup

Training environment and datasets

Dataset: AFHQ v2

- High-quality animal faces (Cat, Dog, Wild).
- ~15,000 images, resized to 256×256 .
- **Preprocessing:** PiDiNet Safe Mode edges + VAE Latents ($32 \times 32 \times 4$).

Training Configuration

- **Hardware:** Single NVIDIA RTX 4070 (12GB VRAM). demonstrating the efficiency of the method.
- **Duration:** Trained for ~480 epochs (Batch size 64, Accumulation 4).

Evaluation Metrics

- Qualitative: Visual fidelity and edge alignment.
- Diagnostic: Training Loss (MSE) and Velocity Magnitude Ratio (RMS_{pred}/RMS_{target}).

Training Dynamics & Diagnosis

Analyzing the "Velocity Undershoot" phenomenon

Convergence

- Raw MSE dropped significantly from 5.81 to 2.53 (-56%), indicating the model is effectively learning the flow field.

The "Velocity Undershoot" Discovery

- We monitored the RMS ratio: $R = RMS(u_{pred}) / RMS(u_{target})$.
- Observation: The ratio improved from 0.05 to 0.55, but stabilized < 1.0 .
- Interpretation:** The model tends to be "conservative," predicting average velocities with smaller magnitudes than the ground truth. This suggests a trade-off where the model minimizes variance by undershooting the target displacement.

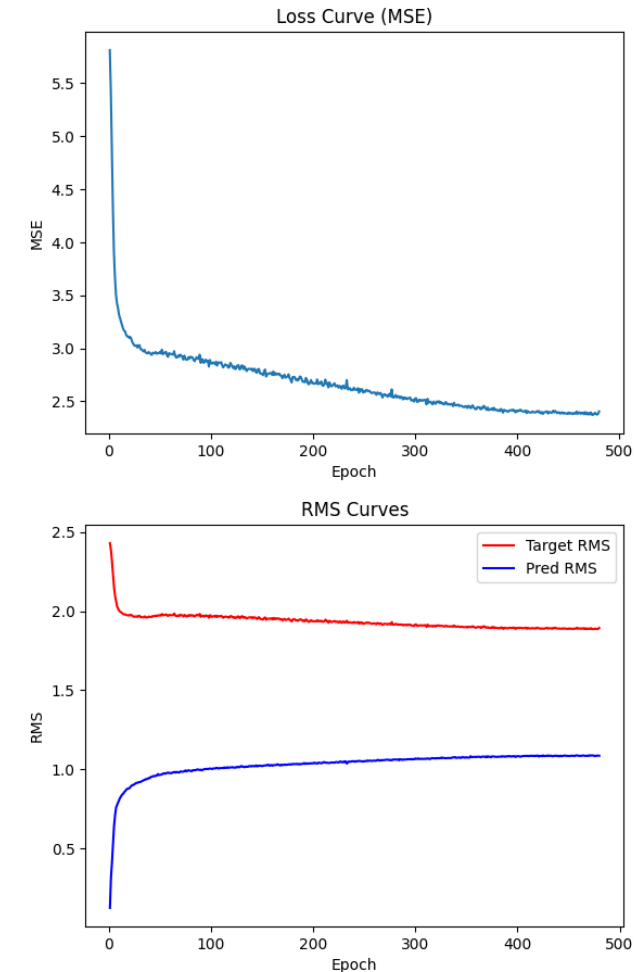


Fig 6: Training log

Qualitative Results: 1-NFE Generation

High-fidelity synthesis in a single forward pass

Cat



Dog



Wild



Edge Input

MeanFlow
1-Step

Ground
Truth



Edge Input

MeanFlow
1-Step

Ground
Truth

Fig 7

Analysis: The Multi-Step Paradox

Why integration accumulates errors instead of quality

Fig 8



Observation: The "Burn" Artifacts

- Contrary to standard diffusion models, increasing NFE (4 steps) introduces severe oversaturation and high-contrast artifacts.
- 1-Step (S1)** produces the most natural result, while **4-Step (S4)** drifts off the data manifold.

Steps	1-NFE	2-NFE	4-NFE
FID	44.06	31.58	28.63

Theoretical Explanation: Non-Conservative Field

- MeanFlow learns a "shortcut" vector field designed specifically for the large $t = 1 \rightarrow 0$ leap.
- This learned field is likely not conservative locally.
- Accumulated Drift:** Breaking the shortcut into small Euler steps accumulates errors from local irregularities in the vector field, leading to the observed divergence.

Impact of Training-Time Guidance

Balancing sharpness and flow consistency

Quantitative Comparison

CFG Scale (ω)	Pred RMS	Target RMS	Ratio	Visual Effect
1.0 (No CFG)	0.99	1.28	0.78	Consistent but blurry
2.0 (Standard)	1.06	1.91	0.55	Sharper, structural drift

- **The Trade-off**
 - Higher ω (2.0): Increases the magnitude of the target velocity, making the regression task harder (lower Ratio), but results in perceptually sharper edges.
 - Lower ω (1.0): Easier to learn (higher Ratio), but images lack high-frequency details.
- Decision: We use $\omega = 2.0$ as the default to prioritize perceptual sharpness for edge alignment.

Summary of Contributions

Delivering a functional Edge-to-Image MeanFlow framework

1. Independent MeanFlow Implementation

- Successfully reconstructed the full training framework from mathematical derivations (JVP identity, Training-time CFG) without official code.

2. Edge-to-Image Extension **[Novelty]**

- Proposed and validated a **Multi-scale Edge Encoder** with Zero-Conv injection, successfully bridging the resolution gap ($256 \rightarrow 32$).

3. Data-Centric Optimization **[Novelty]**

- Demonstrated that PiDiNet (Safe Mode) significantly outperforms traditional operators for animal face generation.

4. Efficiency Demonstrated

- Validated that high-quality generation is possible with 1-NFE on a single consumer GPU (RTX 4070).

Limitations & Critical Insights

Understanding the boundaries of current performance

1. Velocity Magnitude Undershoot

- **Observation:** The RMS ratio stabilizes at ≈ 0.55 , indicating the model consistently predicts smaller displacement vectors than required.
- **Impact:** Results in slightly softer textures compared to ground truth.

2. The Multi-Step Paradox

- **Observation:** Increasing sampling steps (2 or 4) degrades quality ("burn" artifacts).
- **Insight:** The learned vector field is a specialized "1-step shortcut" that does not support robust multi-step integration.

3. Dataset Scale

- **Context:** Trained on AFHQ v2 (~15K images).
- **Limitation:** While successful for faces, the model has not yet been stress-tested on diverse, in-the-wild scenes (e.g., COCO).

Key References

- [1] **MeanFlow (Core Theory)**: Geng et al., "Mean Flows for One-step Generative Modeling", NeurIPS, 2025.
- [2] **Flow Matching**: Lipman et al., "Flow Matching for Generative Modeling", ICLR 2023.
- [3] **ControlNet (Architecture)**: Zhang et al., "Adding Conditional Control to Text-to-Image Diffusion Models", ICCV 2023.
- [4] **Consistency Models**: Song et al., "Consistency Models", ICML 2023.
- [5] **PiDiNet**: Su et al., "Pixel Difference Networks for Efficient Edge Detection", ICCV 2021.
- [6] **AFHQ v2 Dataset**: Choi et al., "StarGAN v2: Diverse Image Synthesis for Multiple Domains", CVPR 2020.

Thank you for your watching